

Matrices and Matrix Operations in MATLAB

MATLAB Fundamentals

A detailed teaching note for beginners

Prepared for students learning introductory MATLAB programming

March 19, 2026

Contents

1	Introduction	3
2	What is a Matrix?	3
2.1	Important Terminology	3
3	Creating Matrices in MATLAB	4
3.1	Example 1: A Simple Matrix	4
3.2	Example 2: Another Matrix	4
3.3	Explanation	4
4	Special Matrices in MATLAB	4
4.1	Zero Matrix	4
4.2	Ones Matrix	5
4.3	Identity Matrix	5
4.4	Random Matrix	5
5	Size of a Matrix	5
6	Accessing Elements of a Matrix	6
6.1	Selecting Entire Rows or Columns	6
6.2	Extracting a Submatrix	6
7	Changing Elements in a Matrix	7
8	Matrix Addition and Subtraction	7
8.1	Example	7
8.2	Explanation	7
9	Scalar Multiplication	7
10	Matrix Multiplication	8
10.1	Important Rule	8
10.2	Example	8
10.3	Explanation	8
11	Element-wise Multiplication and Division	8
11.1	Element-wise Multiplication	8
11.2	Element-wise Division	9
11.3	Element-wise Powers	9
12	Transpose of a Matrix	9
13	Determinant of a Matrix	10
14	Inverse of a Matrix	10
15	Solving Systems of Linear Equations	10

16 Matrix Concatenation	11
16.1 Horizontal Concatenation	11
16.2 Vertical Concatenation	11
17 Sum, Mean, Maximum, and Minimum	11
17.1 Explanation	12
18 Useful MATLAB Matrix Functions	12
19 Worked Examples	12
19.1 Example 1: Create and Inspect a Matrix	12
19.2 Example 2: Matrix Addition	13
19.3 Example 3: Matrix Multiplication	13
19.4 Example 4: Row and Column Extraction	13
19.5 Example 5: Solve a Linear System	13
20 Common Mistakes Students Should Avoid	13
21 Self Exercises	13
22 Challenge Tasks	15
23 Conclusion	16

1 Introduction

Matrices are one of the most important concepts in MATLAB. In fact, the name MATLAB stands for **MATrix LABoratory**, which already shows how central matrices are in the software.

A matrix is a rectangular arrangement of numbers written in rows and columns. Matrices are used in:

- engineering calculations,
- scientific computing,
- computer graphics,
- data analysis,
- machine learning, and
- solving systems of equations.

In MATLAB, a matrix is stored as a two-dimensional array. Learning how to create matrices and perform operations on them is a major step toward understanding numerical computing.

2 What is a Matrix?

A matrix is a collection of values arranged in rows and columns.

For example,

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

This matrix has:

- 2 rows
- 3 columns

So we say that matrix A is of size:

$$2 \times 3$$

2.1 Important Terminology

- **Row:** a horizontal set of elements.
- **Column:** a vertical set of elements.
- **Element:** an individual value inside the matrix.
- **Dimension or size:** the number of rows and columns.

3 Creating Matrices in MATLAB

In MATLAB, matrices are created using square brackets.

- Separate elements in the same row using spaces or commas.
- Separate rows using semicolons.

3.1 Example 1: A Simple Matrix

Listing 1: Creating a matrix in MATLAB

```
1 A = [1 2 3; 4 5 6; 7 8 9];
```

This creates the matrix:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

3.2 Example 2: Another Matrix

```
1 B = [2, 4; 6, 8];
```

This gives:

$$B = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$$

3.3 Explanation

Each row must have the same number of elements. MATLAB does not allow matrices with uneven row lengths.

For example, the following is invalid:

```
1 C = [1 2 3; 4 5];
```

because the first row has 3 elements and the second row has only 2.

4 Special Matrices in MATLAB

MATLAB provides built-in functions for generating useful matrices quickly.

4.1 Zero Matrix

A zero matrix contains only zeros.

Listing 2: Creating a zero matrix

```
1 Z = zeros(3,4);
```

This creates a 3×4 matrix of zeros.

$$Z = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

4.2 Ones Matrix

Listing 3: Creating a matrix of ones

```
1 O = ones(2,3);
```

This creates:

$$O = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

4.3 Identity Matrix

An identity matrix is a square matrix with 1s on the main diagonal and 0s elsewhere.

Listing 4: Creating an identity matrix

```
1 I = eye(3);
```

This gives:

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

4.4 Random Matrix

Listing 5: Creating random matrices

```
1 R1 = rand(2,3);           % random values between 0 and 1
2 R2 = randi(10,3,3);       % random integers from 1 to 10
```

5 Size of a Matrix

To know the size of a matrix, MATLAB uses the `size` function.

Listing 6: Finding matrix size

```
1 A = [1 2 3; 4 5 6];
2 s = size(A);
```

This returns:

$$[2 \ 3]$$

meaning 2 rows and 3 columns.

You can also extract rows and columns separately:

```
1 [rows, cols] = size(A);
```

6 Accessing Elements of a Matrix

MATLAB uses **1-based indexing**. This means counting starts from 1.

If

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

then:

- $A(1,1)$ is 1
- $A(2,3)$ is 6
- $A(3,2)$ is 8

Listing 7: Accessing matrix elements

```

1 A = [1 2 3; 4 5 6; 7 8 9];
2
3 x = A(1,1);    % first row, first column
4 y = A(2,3);    % second row, third column
5 z = A(3,2);    % third row, second column

```

6.1 Selecting Entire Rows or Columns

Listing 8: Selecting rows and columns

```

1 A = [1 2 3; 4 5 6; 7 8 9];
2
3 row2 = A(2,:);    % second row
4 col3 = A(:,3);    % third column

```

Explanation:

- The colon `:` means “all elements” in that row or column direction.
- $A(2,:)$ means row 2, all columns.
- $A(:,3)$ means all rows, column 3.

6.2 Extracting a Submatrix

Listing 9: Extracting part of a matrix

```

1 A = [1 2 3; 4 5 6; 7 8 9];
2
3 B = A(1:2, 2:3);

```

This gives:

$$B = \begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix}$$

7 Changing Elements in a Matrix

You can modify individual elements or entire rows and columns.

Listing 10: Modifying matrix elements

```

1 A = [1 2 3; 4 5 6; 7 8 9];
2
3 A(2,2) = 50;      % change the middle element
4 A(1,:) = [10 20 30]; % replace first row
5 A(:,3) = [100; 200; 300]; % replace third column

```

8 Matrix Addition and Subtraction

Two matrices can be added or subtracted only if they have the same dimensions.

$$A + B$$

and

$$A - B$$

are only defined when A and B have the same number of rows and columns.

8.1 Example

Listing 11: Matrix addition and subtraction

```

1 A = [1 2; 3 4];
2 B = [5 6; 7 8];
3
4 C = A + B;
5 D = B - A;

```

Results:

$$C = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix} \quad D = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}$$

8.2 Explanation

Addition and subtraction happen element by element:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix}$$

9 Scalar Multiplication

A scalar is a single number. Multiplying a matrix by a scalar multiplies every element by that number.

Listing 12: Scalar multiplication

```

1 A = [1 2; 3 4];
2 B = 3 * A;

```


This gives:

$$B = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}$$

10 Matrix Multiplication

Matrix multiplication is different from element-wise multiplication.

If matrix A has size $m \times n$ and matrix B has size $n \times p$, then the product AB is possible and will have size $m \times p$.

10.1 Important Rule

For

$$A \times B$$

to be defined:

$$\text{number of columns of } A = \text{number of rows of } B$$

10.2 Example

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Listing 13: Matrix multiplication

```
1 A = [1 2; 3 4];
2 B = [5 6; 7 8];
3
4 C = A * B;
```

Result:

$$C = \begin{bmatrix} 1(5) + 2(7) & 1(6) + 2(8) \\ 3(5) + 4(7) & 3(6) + 4(8) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

10.3 Explanation

Each element in the result is found by multiplying rows of the first matrix by columns of the second matrix.

11 Element-wise Multiplication and Division

Sometimes we want to multiply corresponding elements one by one instead of doing full matrix multiplication. MATLAB uses the dot operator for this.

11.1 Element-wise Multiplication

Listing 14: Element-wise multiplication

```
1 A = [1 2; 3 4];
2 B = [5 6; 7 8];
3
4 C = A .* B;
```

This gives:

$$C = \begin{bmatrix} 1 \cdot 5 & 2 \cdot 6 \\ 3 \cdot 7 & 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

11.2 Element-wise Division

Listing 15: Element-wise division

```
1 A = [10 20; 30 40];
2 B = [2 4; 5 8];
3
4 C = A ./ B;
```

This gives:

$$C = \begin{bmatrix} 5 & 5 \\ 6 & 5 \end{bmatrix}$$

11.3 Element-wise Powers

Listing 16: Element-wise powers

```
1 A = [1 2; 3 4];
2 B = A.^2;
```

This gives:

$$B = \begin{bmatrix} 1 & 4 \\ 9 & 16 \end{bmatrix}$$

12 Transpose of a Matrix

The transpose of a matrix changes rows into columns and columns into rows.

If

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

then

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

In MATLAB:

Listing 17: Transpose of a matrix

```
1 A = [1 2 3; 4 5 6];
2 AT = A';
```

13 Determinant of a Matrix

The determinant is a special value that can be computed only for square matrices.

For a 2×2 matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

the determinant is:

$$\det(A) = ad - bc$$

In MATLAB:

Listing 18: Determinant of a matrix

```
1 A = [2 3; 1 4];
2 d = det(A);
```

Since:

$$\det(A) = (2)(4) - (3)(1) = 8 - 3 = 5$$

14 Inverse of a Matrix

The inverse of a square matrix A is written as A^{-1} . It exists only if the determinant is not zero.

In MATLAB:

Listing 19: Inverse of a matrix

```
1 A = [2 1; 5 3];
2 A_inv = inv(A);
```

Important note: In practical MATLAB work, solving systems with the backslash operator is usually better than computing the inverse directly.

15 Solving Systems of Linear Equations

One powerful use of matrices is solving equations.

Consider the system:

$$2x + y = 5$$

$$x + 3y = 6$$

This can be written in matrix form as:

$$AX = B$$

where

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}, \quad X = \begin{bmatrix} x \\ y \end{bmatrix}, \quad B = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

In MATLAB, solve it using:

Listing 20: Solving a system of equations

```
1 A = [2 1; 1 3];  
2 B = [5; 6];  
3  
4 X = A \ B;
```

The result stored in `X` contains the values of x and y .

16 Matrix Concatenation

Concatenation means joining matrices together.

16.1 Horizontal Concatenation

Listing 21: Joining matrices side by side

```
1 A = [1 2; 3 4];  
2 B = [5 6; 7 8];  
3  
4 C = [A B];
```

This gives:

$$C = \begin{bmatrix} 1 & 2 & 5 & 6 \\ 3 & 4 & 7 & 8 \end{bmatrix}$$

16.2 Vertical Concatenation

Listing 22: Joining matrices top to bottom

```
1 A = [1 2; 3 4];  
2 B = [5 6; 7 8];  
3  
4 C = [A; B];
```

This gives:

$$C = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}$$

17 Sum, Mean, Maximum, and Minimum

MATLAB can perform useful calculations on matrix data.

Listing 23: Basic matrix statistics

```
1 A = [1 2 3; 4 5 6; 7 8 9];  
2  
3 s = sum(A);  
4 m = mean(A);  
5 mx = max(A);  
6 mn = min(A);
```

17.1 Explanation

By default, MATLAB works **column-wise** for many of these functions.

For matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

- `sum(A)` gives the sum of each column
- `mean(A)` gives the average of each column
- `max(A)` gives the largest value in each column
- `min(A)` gives the smallest value in each column

18 Useful MATLAB Matrix Functions

Function	Purpose
<code>size(A)</code>	Returns matrix dimensions
<code>length(A)</code>	Returns largest dimension
<code>zeros(m,n)</code>	Creates an $m \times n$ zero matrix
<code>ones(m,n)</code>	Creates an $m \times n$ ones matrix
<code>eye(n)</code>	Creates an identity matrix
<code>rand(m,n)</code>	Creates a random matrix
<code>det(A)</code>	Computes determinant
<code>inv(A)</code>	Computes inverse
<code>A'</code>	Transpose of matrix
<code>sum(A)</code>	Sums each column
<code>mean(A)</code>	Mean of each column

19 Worked Examples

19.1 Example 1: Create and Inspect a Matrix

```

1 A = [2 4 6; 8 10 12];
2 size(A)
3 A(2,3)
```

This example:

- creates a 2×3 matrix,
- checks its size,
- selects the element in row 2, column 3.

19.2 Example 2: Matrix Addition

```
1 A = [1 2; 3 4];  
2 B = [10 20; 30 40];  
3 C = A + B;
```

19.3 Example 3: Matrix Multiplication

```
1 A = [1 2; 3 4];  
2 B = [2 0; 1 2];  
3 C = A * B;
```

19.4 Example 4: Row and Column Extraction

```
1 A = [5 10 15; 20 25 30; 35 40 45];  
2  
3 r = A(1,:);  
4 c = A(:,2);
```

19.5 Example 5: Solve a Linear System

```
1 A = [3 2; 1 4];  
2 B = [18; 16];  
3 X = A \ B;
```

20 Common Mistakes Students Should Avoid

1. Forgetting that MATLAB indexing starts at 1, not 0.
2. Using `*` when element-wise multiplication `.*` is needed.
3. Trying to add or subtract matrices of different sizes.
4. Attempting matrix multiplication when dimensions do not match.
5. Confusing `A(:,2)` with `A(2,:)`.
6. Using `inv(A)` unnecessarily instead of `A \ B` for solving equations.
7. Forgetting that many MATLAB functions work column-wise by default.

21 Self Exercises

Exercise Set A: Basic Practice

1. Create the matrix

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

2. Create a 3×3 zero matrix.
3. Create a 4×2 matrix of ones.
4. Create a 5×5 identity matrix.

5. For

$$A = \begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \end{bmatrix}$$

find:

- (a) the size of A
 - (b) the element in row 2, column 3
 - (c) the first row
 - (d) the second column
6. Let

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

compute:

- (a) $A + B$
 - (b) $B - A$
 - (c) $A .* B$
 - (d) $A * B$
7. Transpose the matrix

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

8. Find the determinant of

$$\begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}$$

9. Extract the submatrix from rows 1 to 2 and columns 2 to 3 of

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

10. Solve the system:

$$x + 2y = 7$$

$$3x + 4y = 18$$

using MATLAB matrix methods.

Exercise Set B: Thinking Questions

1. What is the difference between a row vector, a column vector, and a matrix?
2. Why must two matrices have the same dimensions before they can be added?
3. Why is matrix multiplication different from element-wise multiplication?
4. What does the transpose of a matrix do?
5. Why is the backslash operator often preferred for solving systems of equations?
6. What does `A(:,3)` mean?
7. What does `A(2,:)` mean?
8. Why must a matrix be square before calculating its determinant?

22 Challenge Tasks

1. Create a 4×4 matrix of random integers from 1 to 20.
2. Replace the third row of a matrix with new values of your choice.
3. Create two compatible matrices and perform:
 - (a) addition
 - (b) subtraction
 - (c) matrix multiplication
 - (d) element-wise multiplication
4. Write a MATLAB script that:
 - (a) creates a matrix
 - (b) displays its size
 - (c) extracts one row and one column
 - (d) computes the transpose
 - (e) computes the sum of each column
5. Use matrices to solve a system of three equations in three unknowns.
6. Create a matrix and then form a new matrix by joining it horizontally with an identity matrix of compatible size.
7. Write MATLAB code that checks whether a matrix is square by comparing the number of rows and columns.

23 Conclusion

Matrices are a central part of MATLAB and a foundation for more advanced programming. Students should be comfortable with:

- defining and creating matrices,
- accessing and modifying matrix elements,
- understanding matrix dimensions,
- performing addition, subtraction, and scalar multiplication,
- distinguishing between matrix multiplication and element-wise multiplication,
- using transpose, determinant, and inverse,
- extracting rows, columns, and submatrices, and
- solving systems of equations using matrices.

Once these ideas are understood well, students will be ready for more advanced topics such as matrix algebra, plotting data from matrices, numerical methods, and scientific computing.